Markdown Editor – 仕様・開発環境ドキュメント

> 作成日: 2026-03-02

1. プロジェクト概要

項目	内容
アプリ名	Markdown Editor
種別	ローカル動作デスクトップアプリケーション
対応OS	Windows / Linux
アーキテクチャ	フロントエンド・バックエンド分離型（Tauri IPC）

処理速度に優れたローカル動作のマークダウンエディタ。将来的なクラウド（Web）展開を見据え、React フロントエンドと Rust バックエンドを IPC で明確に分離した設計を採用。

2. 機能要件

2.1 画面レイアウト（3ペイン構成）

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│  Header: アプリ名 │ 開いているファイル名 │ アクションボタン   │
├──────────────────┬──────────────────────────────────┬──────────────────────────────────────┤
│                  │                                  │                                      │
│ ファイル   │    エディタ（中央）      │    プレビュー（右）       │
│ ツリー     │   マークダウン入力      │   HTML リアルタイム     │
│ （左20%）  │       （40%）           │   レンダリング（40%）   │
│                  │                                  │                                      │
└──────────────────┴──────────────────────────────────┴──────────────────────────────────────┘
```

2.2 機能一覧

ヘッダー

No.	機能	詳細
H-1	ファイル名表示	開いているファイル名を表示
H-2	未保存インジケータ	変更がある場合、ファイル名の横に黄色ドットを表示
H-3	フォルダを開く	ダイアログでローカルディレクトリを選択し、ファイルツリーに表示
H-4	新規作成	新規ファイルの保存先をダイアログで指定して作成・開く
H-5	保存	現在のファイルを上書き保存（未変更時は非活性）
H-6	PDF 出力	プレビューペインの内容を OS の印刷ダイアログ経由で PDF 保存

左ペイン（ファイルツリー）

No.	機能	詳細
L-1	ディレクトリ表示	選択したフォルダ配下のファイル・フォルダをツリー表示（最大3階層）
L-2	フォルダ展開	クリックでフォルダの展開/折りたたみ
L-3	ファイルを開く	.md ファイルをクリックすると中央エディタに内容を読み込む
L-4	ソート	フォルダ優先・名前昇順で自動ソート
L-5	隠しファイル除外	`.` で始まるファイル・フォルダは非表示
L-6	アクティブ強調	現在開いているファイルをハイライト表示

中央ペイン（エディタ）

No.	機能	詳細
C-1	マークダウン入力	テキストエリアへの自由入力
C-2	タブ補完	Tab キーで 2 スペースを挿入（フォーカス移動しない）
C-3	モノスペースフォント	JetBrains Mono / Fira Code / Consolas 等を使用
C-4	キーボードショートカット	**Ctrl+S** でファイル保存

右ペイン（プレビュー）

No.	機能	詳細
P-1	リアルタイムプレビュー	入力と同時にマークダウンを HTML レンダリング
P-2	GFM 対応	テーブル・チェックボックス・取り消し線などの GitHub Flavored Markdown
P-3	シンタックスハイライト	コードブロックを highlight.js でハイライト（github-dark テーマ）
P-4	PDF 印刷スタイル	印刷時はプレビュー内容のみを白背景で出力

2.3 IPC コマンド（Rust バックエンド）

コマンド	引数	戻り値	役割
`read_dir`	`path: &str`	`Vec<FileInfo>`	ディレクトリを再帰的に読み込み、ファイル一覧を返す
`read_file`	`path: &str`	`String`	テキストファイルの内容を読み込む
`save_file`	`path: &str, content: &str`	`()`	ファイルを上書き保存（親ディレクトリも自動生成）

```
// FileInfo 構造体（JSON シリアライズ）
struct FileInfo {
    name: String,
    path: String,
    is_dir: bool,
    children: Option<Vec<FileInfo>>,
}
```

---

3. 非機能要件

区分	要件
パフォーマンス	ファイル読み書きは Rust で処理し、低レイテンシを実現
オフライン動作	完全ローカル動作・ネットワーク不要
拡張性	フロントエンド・バックエンド分離により将来的な Web 展開が容易

区分	要件
セキュリティ	Tauri CSP・ケイパビリティ制御で IPC アクセスを最小権限に制限
ウィンドウ	初期サイズ 1400×900、最小サイズ 900×600

---

4. 技術スタック

フロントエンド

ライブラリ	バージョン	用途
React	19.1.0	UI フレームワーク
TypeScript	5.8.3	型安全な開発
Vite	7.0.4	ビルドツール・開発サーバー
Tailwind CSS	4.2.1	スタイリング
@tailwindcss/vite	4.2.1	Vite 向け Tailwind プラグイン
react-markdown	10.1.0	マークダウン → HTML 変換
remark-gfm	4.0.1	GitHub Flavored Markdown サポート
rehype-highlight	7.0.2	シンタックスハイライト統合
highlight.js	11.11.1	コードハイライトエンジン

バックエンド（Rust）

クレート	バージョン	用途
tauri	2	デスクトップアプリフレームワーク
tauri-build	2	ビルドスクリプト
tauri-plugin-opener	2	URL・ファイルオープン
tauri-plugin-dialog	2	ファイル・フォルダ選択ダイアログ
tauri-plugin-shell	2	シェルコマンド実行

クレート	バージョン	用途
serde	1	JSON シリアライズ（derive）
serde_json	1	JSON 処理

Tauri プラグイン（フロントエンド）

パッケージ	バージョン
@tauri-apps/api	^2
@tauri-apps/plugin-dialog	2.6.0
@tauri-apps/plugin-shell	2.3.5
@tauri-apps/plugin-opener	^2

---

5. 開発環境

必須ツール

ツール	バージョン	備考
Node.js	v24.13.1	フロントエンドビルド
npm	11.8.0	パッケージ管理
Rust	1.93.1 (2026-02-11)	バックエンドコンパイル
Cargo	1.93.1	Rust パッケージ管理
rustup	-	Rust ツールチェイン管理

Rust インストール（Windows）

```
# rustup-init.exe をダウンロードして実行
Invoke-WebRequest -Uri "https://static.rust-lang.org/rustup/dist/x86_64-pc-windows-msvc/rustup-ini
& "$env:TEMP¥rustup-init.exe" -y --default-toolchain stable
```

開発コマンド

```
# 依存関係のインストール（初回）
npm install

# 開発サーバー起動（ホットリロード付き）
npm run tauri dev

# プロダクションビルド
npm run tauri build

# フロントエンドのみビルド確認
npx vite build

# Rust バックエンドのコンパイル確認のみ
cd src-tauri && cargo check
```

プロジェクト構成

```
markdown_previewApp/
├── src/                        # React フロントエンド
│   ├── main.tsx                 # エントリーポイント
│   ├── App.tsx                  # メインレイアウト・状態管理
│   ├── index.css                # Tailwind CSS v4 + テーマ定義
│   └── components/
│       ├── Header.tsx           # ヘッダー・アクションボタン
│       ├── FileTree.tsx         # ファイルツリー
│       ├── Editor.tsx           # マークダウンエディタ
│       └── Preview.tsx          # リアルタイムプレビュー
├── src-tauri/                  # Rust バックエンド
│   ├── src/
│   │   ├── main.rs               # エントリーポイント
│   │   └── lib.rs                # IPC コマンド実装
│   ├── Cargo.toml               # Rust 依存関係
│   ├── tauri.conf.json          # Tauri 設定（ウィンドウ・バンドル）
│   └── capabilities/
│       └── default.json         # IPC パーミッション設定
├── test_docs/                  # テスト用マークダウンファイル
│   └── sample.md
├── vite.config.ts              # Vite + Tailwind 設定
├── package.json
├── tsconfig.json
└── SPECIFICATION.md            # 本ドキュメント
```

---

6. PDF エクスポート方式

外部バイナリ（Chrome ヘッドレス・wkhtmltopdf 等）への依存を排除するため、ブラウザ標準の `window.print()` を採用。

```
① ユーザーが「PDF」ボタンをクリック
      ↓
② window.print() を呼び出し
      ↓
③ CSS @media print により
   プレビューペインのみを白背景で表示
      ↓
④ OS の印刷ダイアログが開く
      ↓
⑤ 印刷先で「PDF に保存」を選択して出力
```

メリット: 追加依存ゼロ・クロスプラットフォーム対応・CSS で印刷スタイルを完全制御可能

ドキュメント生成: Antigravity Agent – 2026-03-02